

VIETNAM NATIONAL UNIVERSITY HO CHI MINH CITY
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY
FACULTY OF COMPUTER SCIENCE AND ENGINEERING



Web Programming

Semester Project Report

Restaurant & Food Ordering Website

Advisor(s): Nguyen Duc Thai

Student(s): Nguyen Thanh Binh 2252082

HO CHI MINH CITY, MAY 2026



Contents

1	Introduction and Purpose	4
2	System Requirements	4
2.1	Functional Requirements	4
2.2	Non-Functional Requirements	5
3	Design and Layout Decisions	6
3.1	Domain-Specific Design Choices	6
3.2	Layout and Responsive Design	6
4	Database Schema	7
4.1	Data Integrity Decisions	7
5	Implementation Details	8
5.1	Dynamic Search with AJAX	8
5.2	User Authentication	8
5.3	Menu Catalog & Store Locations	8
5.4	Search Engine Optimization (SEO)	8
6	Challenges Faced and Lessons Learned	9

List of Figures

List of Tables

2.1	Functional Requirements	5
4.1	Optimized Database Schema and Constraints	7



1 Introduction and Purpose

This report details the design and implementation of a dynamic Restaurant / Food Ordering Website, developed as the final semester project for the Web Programming course. The primary purpose of this web application is to provide a seamless digital menu experience where users can browse restaurant offerings, search for specific dishes in real-time, view store availability, and manage their personal accounts.

The project applies core web technologies including HTML/CSS, JavaScript (AJAX), PHP, and MySQL to deliver a structured, responsive, and database-driven platform.

Project Github: [Restaurant Management](#)

2 System Requirements

2.1 Functional Requirements

The core functionalities of the Restaurant / Food Ordering Website are designed to fulfill the mandatory project specifications[cite: 7]:

ID	Feature	Description
FR1	User Authentication	The system must allow users to register, log in, log out, and recover forgotten passwords[cite: 7].
FR2	Menu Catalog	The website must display items from a MySQL database, supporting sorting (e.g., by price or name) and pagination (or lazy loading)[cite: 7].
FR3	Dynamic Search	An AJAX-powered search function must update results dynamically while the user types, without requiring a full page reload[cite: 7].
FR4	Categories & Navigation	Menu items must be organized into categories, and relevant pages must display breadcrumb navigation (e.g., Home > Products > Category > Item)[cite: 7].
FR5	Store Locations & Maps	The detail page for each item must display the physical restaurant branches where it is available, along with direct Google Maps links[cite: 7].

Table 2.1: Functional Requirements

2.2 Non-Functional Requirements

To ensure usability, security, and performance, the system adheres to the following non-functional constraints[cite: 7]:

- **Responsive Design:** The application must provide a seamless user interface across desktop, tablet, and mobile devices using flexible layouts (e.g., Bootstrap)[cite: 7].
- **Security:** User passwords must be securely hashed (never stored in plain text), and all user inputs must be validated (e.g., email format, password length)[cite: 7].
- **Search Engine Optimization (SEO):** The site must implement at least three SEO techniques, such as semantic HTML elements, meaningful meta descriptions, and friendly URLs[cite: 7].
- **Technology Stack:** The system must be dynamic and database-driven using MySQL for data storage and PHP for server-side logic[cite: 7].



3 Design and Layout Decisions

3.1 Domain-Specific Design Choices

To cater to the restaurant domain, the UI was designed to highlight food imagery and simplify navigation. Items are grouped logically into categories (e.g., Appetizers, Main Courses, Drinks), and a breadcrumb navigation system (e.g., *Home > Items > Search > Contact*) is implemented to help users track their location within the catalog.

3.2 Layout and Responsive Design

The website features a flexible layout designed to adapt to desktop, tablet, and mobile devices using Bootstrap 5.

- **Header:** Features a left-aligned restaurant logo and a Navigation Bar containing links to Home, Items, Search, and Contact.
- **Main Content:** Utilizes a flexible grid system to display menu items in visually appealing cards.
- **Footer:** Contains copyright information, quick links, and contact details.



4 Database Schema

The application uses a MySQL relational database to manage dynamic content. Based on database best practices, the schema enforces strict data types, unique constraints, and auto-incrementing primary keys to ensure data integrity and prevent frontend errors.

Table Name	Columns, Data Types & Constraints
users	id : INT, Primary Key, AUTO_INCREMENT email : VARCHAR(255), UNIQUE, NOT NULL password : VARCHAR(255), NOT NULL (<i>Accommodates bcrypt hashing</i>) created_at : TIMESTAMP, DEFAULT CURRENT_TIMESTAMP categories : VARCHAR(255), Nullable
items	id : INT, Primary Key, AUTO_INCREMENT name : VARCHAR(255), NOT NULL price : DECIMAL(10,2), NOT NULL (<i>Supports currency formatting</i>) description : TEXT, Nullable category_id : INT, Nullable, Foreign Key

Table 4.1: Optimized Database Schema and Constraints

4.1 Data Integrity Decisions

To optimize the database from its initial draft, several constraints were applied:

- **Primary Keys:** AUTO_INCREMENT was added to all id columns so the database handles unique record identification automatically.
- **Currency Handling:** The price column was strictly defined as DECIMAL(10,2) rather than a standard integer or float, ensuring accurate calculation of monetary values up to two decimal places.
- **Nullability Constraints:** Critical fields such as name, price, email, and password were restricted with NOT NULL to prevent incomplete data insertions that could break the application's user interface.



5 Implementation Details

5.1 Dynamic Search with AJAX

To enhance user experience, a real-time search feature was implemented using AJAX. As the user types a query into the search bar, JavaScript fetches matching items from the PHP backend and dynamically updates the DOM. This prevents full page reloads and allows users to instantly click on a result to view item details.

5.2 User Authentication

A robust authentication system allows users to Register, Login, Logout, and recover forgotten passwords. Security measures include:

- **Input Validation:** Ensuring emails are formatted correctly and passwords meet minimum length requirements.
- **Password Hashing:** Using PHP's native `password_hash()` to prevent plain-text storage of credentials.

5.3 Menu Catalog & Store Locations

The menu catalog loads data dynamically from MySQL. It supports sorting (by Name and Price) and utilizes **[Insert your choice: Pagination OR Lazy Loading]** to manage large amounts of data efficiently. Furthermore, individual item pages query the database to display specific restaurant locations where the dish is available, complete with clickable Google Maps links.

5.4 Search Engine Optimization (SEO)

Three core SEO practices were applied to improve search visibility:

1. **Semantic HTML:** Utilizing `<header>`, `<nav>`, `<main>`, and `<article>` tags for better screen reader and crawler parsing.
2. **Meaningful Meta Tags:** Including unique page titles and descriptive meta descriptions for each menu category.
3. **Friendly URLs:** Utilizing URL rewriting to create readable paths (e.g., `/items/spring-rolls` instead of `item.php?id=3`).



6 Challenges Faced and Lessons Learned

Throughout the development of this semester project, several technical and structural hurdles provided valuable learning opportunities:

- **Project Organization and Architecture:** Transitioning from a flat-file structure to a strict Model-View-Controller (MVC) pattern was a significant hurdle. Initially, mixing backend PHP logic directly with HTML views resulted in broken asset paths and difficult-to-maintain code. Learning to implement a "Front Controller" (`index.php`) to securely route all web traffic and isolate database credentials in a hidden `config/` directory was challenging. However, this struggle provided hands-on experience with enterprise-level project structuring and security best practices.
- **Store Locations and Maps Integration:** Implementing the physical store locations and Google Maps integration proved to be the most difficult functional requirement. Attempting to dynamically query location addresses from the database and format them into interactive Google Maps URLs required handling external links and complex SQL JOINS. Navigating this requirement highlighted the steep learning curve associated with integrating third-party services and mapping APIs, leading to a simplified implementation for the final demonstration.
- **AJAX Integration:** Handling asynchronous JavaScript requests and parsing JSON responses from the PHP backend without refreshing the page was initially confusing. Learning to debug network requests in the browser console to build the live-search feature was a crucial takeaway for modern frontend development.
- **Data Integrity and Security:** Implementing secure password hashing (`password_hash`) and properly managing PHP sessions highlighted the critical difference between client-side form validation and true server-side security.